Session 6 · Deck 6.B1

Krusell–Smith: When the Distribution Is a State

Zhigang Feng

Thursday, October 29, 2026

© Zhigang Feng • Redistribution prohibited without permission

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 6.B1 ▼

S1 ✓	S2 ✓	S3 ✓	S4 ✓	S5 ✓	S6	S7	S8	S9	S10
Sep 24	Oct 1	Oct 8	Oct 15	Oct 22	Oct 29	Nov 5	Nov 12	Nov 19	Dec 3
Intro	HA · OLG	Num. DP	Numerics	Numerics	HA classic	ML	RL	Text	Agents
Growth	Policy	Python	I	II	The wall	Deep nets	HA + DL	LLMs	Projects
trunk	branches	the machine · classical methods			the wall	new tools		new data	colleagues

Build it on paper → solve it on the machine → push it to the frontier with AI

Block A — Aiyagari: 6.A1 the household, from Bellman to Euler · 6.A2 the population and the price.
Block B: 6.B1 Krusell–Smith · 6.B2 transitions and the life cycle.

© Zhigeng Fang
Redistribution prohibited without permission

Roadmap: Five Parts

2.B1 defined the equilibrium and stopped at "it closes". This deck computes it, by the two routes of 6.A1, and grades the answer.

Part	Topic	The question it answers
1	Two kinds of risk	Why must households forecast the distribution, and what is the bet?
2	The household with two more states	The Bellman equation on (a, ε, K, z), by VFI and by EGM: do they agree?
3	The loop	How do we find a forecasting rule that reproduces itself?
4	Is it right?	"Converged"; R^2 against Den Haan's test; the benchmark.
5	Why it works, where it stops	Why one moment suffices; what the whole distribution costs.

© Zhigang Feng
Reproduction prohibited without permission

Idea

Krusell and Smith's bet: forecast the distribution by its mean. The household then has four states — 6.A1's methods still apply — and the bet is a fixed point in four numbers.

The Economy: Two Kinds of Risk

The economy every comparison in the literature uses: Krusell and Smith's, as parameterized by Den Haan, Judd and Juillard (2010). Quarterly.

Aggregate: productivity $z \in \{0.99,\ 1.01\}$, recession or boom, each lasting 8 quarters on average.

Individual: employed or unemployed. Unemployment is 10% in a recession and 4% in a boom; a spell lasts 2.5 quarters in a recession, 1.5 in a boom. The unemployed get 15% of the wage, paid by a tax on the employed.

Preferences and technology: log utility, $\beta = 0.99$; $Y = zK^{\alpha}L^{1-\alpha}$, $\alpha = 0.36$, $\delta = 0.025$; $a' \geq 0$. Prices depend on (K, z):

$$r = \alpha z (K/L)^{\alpha-1}, \qquad w = (1-\alpha) z (K/L)^{\alpha}.$$

Example: the joint chain, built and checked

The 4×4 matrix over (recession, boom) $\times$ (unemployed, employed) is *built* from the durations plus two restrictions of Krusell and Smith on job finding when z switches:

0.525	0.350	0.031	0.094
0.039	0.836	0.002	0.123
0.094	0.031	0.292	0.583
0.009	0.116	0.024	0.851

It matches the published table to 4×10^{-7}; the code asserts it before running.

Den Haan, Judd & Juillard, *JEDC* 34 (2010), Tables 1–2; Krusell & Smith, *JPE* 106 (1998). Ledger `ks.calibration`, `ks.transition_matrix`.

© Zhigang Feng Redistribution prohibited without permission

Tomorrow's interest rate depends on tomorrow's capital, the mean of tomorrow's distribution: the distribution is a state.

$$V(a,\varepsilon,z,\Gamma) = \max_{0\le a'\le \mathrm{coh}} \Big\{ u(\mathrm{coh}-a') + \beta\,\mathbb{E}\big[V(a',\varepsilon',z',\Gamma') \,\big|\, \varepsilon, z\big] \Big\}$$

Here $\mathrm{coh} = \big(1 + r(K,z) - \delta\big)a + \mathrm{income}(\varepsilon, K, z)$, $K = \int a\, d\Gamma$ and $\Gamma' = H(\Gamma, z, z')$. The household needs r and w tomorrow, so it needs K', so it needs Γ' — the whole distribution, carried forward by the law of motion H (2.B1). No grid can hold Γ as a state variable.

Notation. Γ is the distribution as a state; λ (6.A2) was the invariant one; H is the law of motion households believe.

Idea

In Block A the distribution sat still and prices were numbers. With an aggregate shock that keeps coming, prices are functions of the distribution.

2.B1, "Why the Household Cannot Avoid Forecasting Γ", and "Common Noise Is Why Γ Must Be a State".

© Zhaonan Feng Reproduction prohibited without permission

2.B1's equilibrium with aggregate risk. New since Block A: prices depend on (K, z), and the forecast H is part of the equilibrium.

Definition

A **recursive competitive equilibrium** is a law of motion H, a value V and a policy $a'(a, \varepsilon, z, \Gamma)$, prices $r(K, z)$, $w(K, z)$ and a tax $\tau(z)$ such that:

1. **Households:** given the prices, τ and H, V solves the Bellman equation of the last frame and a' is its policy.
2. **Firm:** $r = \alpha z (K/L_z)^{\alpha-1}$, $w = (1-\alpha) z (K/L_z)^{\alpha}$, with $K = \int a \, d\Gamma$ and L_z the labour employed in state z.
3. **Government:** the tax on the employed pays the benefit (15% of w) every period.
4. **Consistency:** H is what the policy produces: $\Gamma'(\mathcal{B}, \varepsilon') = \sum_{\varepsilon} \pi(\varepsilon' \mid \varepsilon, z, z') \, \Gamma\big(\{a : a'(a, \varepsilon, z, \Gamma) \in \mathcal{B}\}, \varepsilon\big)$ for every set $\mathcal{B}$.

Idea

Condition 4 is the new one: households' beliefs about how the economy moves must match how their own choices move it.

What we compute

An *approximate* equilibrium: condition 4 imposed on a log-linear rule for K (next frame). Part 4 checks what that costs.

© Zhigang Feng Redistribution prohibited without permission

2.B1, "Recursive Competitive Equilibrium with Aggregate Risk"; Krusell & Smith, *JPE* 106 (1998).

Krusell and Smith's Bet: Forecast With the Mean

Replace Γ by its first moment, guess a log-linear law for it, and require the guess to reproduce itself.

$$\log K' = \kappa_0(z) + \kappa_1(z)\,\log K.$$

Four numbers, one line for each aggregate state. The household's state shrinks from $(a, \varepsilon, z, \Gamma)$ to (a, ε, K, z).

The equilibrium requirement. The rule households *use* must be the rule their behaviour *generates*: simulate the economy with it, regress $\log K'$ on $\log K$, and get the same four numbers back. A fixed point in κ.

Example: the converged rule

recession: $\log K' = 0.1240 + 0.9655\,\log K$
boom: $\log K' = 0.1371 + 0.9634\,\log K$

Left alone, the rule settles at $K = 36.4$ in a long recession and 42.4 in a long boom.

Pitfall

The bet is checked, not assumed: Part 4 grades the rule, Part 5 says when it fails.

© Zhigang Feng • Redistribution prohibited without permission

Ledger `ks.solution` (`kappa_bad`, `kappa_good`), `ks.diagnostics.K` (`rule_rest_bad`, `rule_rest_good`).

The Function Now Has Four Arguments

Given the rule, the aggregate state enters only through prices today and the forecast K' tomorrow. It is 6.A1's problem with a table of prices.

$$V(a,s,K)=\max_{a'}\Big\{u\big(\mathrm{coh}-a'\big)+\beta\sum_{s'}P_{ss'}\,V\big(a',s',K'(K,z)\big)\Big\},$$

with $s=(z,\varepsilon)$ one of four states, $\mathrm{coh}=R(K,z)\,a+\mathrm{inc}(\varepsilon,K,z)$, and $K'(K,z)=\exp\big(\kappa_0(z)+\kappa_1(z)\log K\big)$.

The grid. 120 asset nodes on $[0,300]$, dense near the limit; 7 nodes for K on $[30,50]$, equally spaced in $\log K$; the four values of s. In all $120\times4\times7=3{,}360$ states.

Idea

The joint chain P is 4×4, so the expectation is one matrix product, as in 6.A1. What is new is that K' falls *between* the K-nodes: V must be read there.

Why $\log K$

The rule is linear in $\log K$, so nodes equally spaced in $\log K$ put the forecast the same relative distance from its neighbours everywhere.

© Zhigang Feng · Redistribution prohibited without permission

Ledger `ks.household_methods.state_count`, `ks.settings`.

VFI With Two More States: Interpolate in $\log K$, Then in a

The value route, which Krusell and Smith took: 6.A1's interpolated VFI, with one more interpolation for tomorrow's K.

1: $m_0, \theta \leftarrow$ where $K'(K_m, z)$ falls between K-nodes ▷ linear in $\log K$
2: **repeat**
3: $EV(a_j, s, K_m) \leftarrow$ $\sum_{s'} P_{ss'} \left[(1-\theta) V(a_j, s', K_{m_0}) + \theta V(a_j, s', K_{m_0+1})\right]$
4: each (a_i, s, K_m): $a' \leftarrow \arg\max_x u(\text{coh} - x) + \beta\, EV(x, s, K_m)$ ▷ 6.A1, Part 4
5: $V^{\text{new}} \leftarrow$ the maxima; $d \leftarrow \max |V^{\text{new}} - V|$; $V \leftarrow V^{\text{new}}$
6: Howard: 30 updates of V with a' held fixed ▷ 4.B3
7: **until** $d < 10^{-8}$

Line 3 interpolates in $\log K$ (4.B1, "Two Dimensions"); Line 4 reads EV linearly between asset nodes and maximizes by golden section.

Measured, at the converged rule

Plain VFI (10^{-6}): **1,382** sweeps, 40 s — 1.B2 predicted $\ln 10^{-6} / \ln 0.99 \approx 1{,}375$ for $\beta = 0.99$. With Howard steps: **62** maximizations, 2.3 s.

Pitfall

If K' leaves the K-grid, Line 3 would extrapolate in silence. The code stops instead and asks for a wider grid.

© Zhigang Feng Redistribution prohibited without permission

Krusell & Smith (1998) solved the individual problem by value function iteration. Ledger `ks.household_methods` (`vfi_plain`, `vfi_howard`).

The Euler Route at the Forecast K'

6.A1's EGM with the same $\log K$ weights: tomorrow's consumption is read at the forecast, the Euler equation is inverted, and the limit is a comparison.

repeat
  $c'(a_j, s', K') \leftarrow (1-\theta)\, c(a_j, s', K_{m_0}) + \theta\, c(a_j, s', K_{m_0+1})$
  $E_{jsm} \leftarrow \sum_{s'} P_{ss'}\, R(K', z')\, c'(a_j, s', K')^{-\gamma}$ ▷ one `einsum`
  $\tilde{c} \leftarrow (\beta E)^{-1/\gamma}$; $\tilde{a} \leftarrow (\tilde{c} + a_j - \text{inc})/R$ ▷ invert; endogenous grid
  $c \leftarrow (\tilde{a}, \tilde{c})$ read at the asset nodes; below $\tilde{a}_1$: $c = \text{coh}$ ▷ the limit
until $\max |c^{\text{new}} - c| < 10^{-8}$

Measured, at the converged rule

From scratch: 1,257 sweeps, 0.31 s. Inside the loop of Part 3 it starts from the previous pass's policy.

Idea

Maliar, Maliar and Valli (2010) iterate on the Euler equation on a fixed grid; this code runs it backwards, as 6.A1 did. Same equation, no root-find.

© Zhigang Feng • Redistribution prohibited without permission

Ledger `ks.household_methods.egm`. **Log utility**: $\gamma = 1$.

Two Routes, One Policy

Same household, same rule, same nodes. The two routes agree where the households are, and part only at the top of the grid.

	sweeps	seconds	tolerance
plain VFI	1,382	40	10^{-6} on V
VFI, Howard	62	2.3	10^{-8} on V
EGM	1,257	0.31	10^{-8} on c

Agreement in consumption. Weighted by where households are: 0.11%. Worst below the 99th percentile of wealth ($a = 96$): 2.5%.

Pitfall: the top node

At $a = 300$ the two differ by half of consumption: VFI cannot choose a' above the last node, EGM extrapolates. Nobody lives there — the mass on the top node never exceeds 10^{-10} — which is why the grid reaches three times beyond the richest percent.

The loop of Part 3 calls the household 16 times: EGM's speed is what makes the loop cheap.

Ledger `ks.household_methods` (`agreement`, `wealth_p99`, `mass_at_top_max`); **weights: the time-averaged cross-section and the four states' long-run shares, at the K-node nearest mean capital.**

© Zhigang Feng · Redistribution prohibited without permission

The Loop: Solve, Simulate, Regress, Update

A fixed point in four numbers: the rule households use must equal the rule their behaviour implies.

1: $\kappa \leftarrow$ a guess
2: **repeat**
3: household: $c(a, s, K)$ given κ ▷ Part 2, warm-started
4: simulate: push the histogram 3,000 quarters along one z path; record K_t ▷ 6.A2
5: regress $\log K_{t+1}$ on $\log K_t$, separately for each z, after 500 quarters ▷ $\hat{\kappa}$
6: **if** $\max|\hat{\kappa} - \kappa| < 10^{-5}$ **stop**
7: $\kappa \leftarrow 0.7\,\kappa + 0.3\,\hat{\kappa}$ ▷ damping, 4.A1
8: **until** stopped

`solve_ks` in `s06_ks.py`; **Lab 6, block B1**.

Why damp

A new rule changes behaviour, which changes the rule the regression finds. Taking the full step can overshoot; 30% of it does not (4.A1, "Take Only Part of the Step").

Example

One z path, drawn once with a fixed seed, is reused at every pass: the rule changes, the luck does not.

© Zhigang Feng
Redistribution prohibited without permission

3 · Push a Histogram, Not a Panel — and Never Let a Grid Clamp

Two choices that make the regression trustworthy: no sampling error in the simulated K_t, and no capital outside the grid that prices it.

A histogram. Krusell and Smith simulated a panel of households. This code pushes the distribution over (wealth, employment) on 3,001 nodes 0.1 apart with 6.A2's lottery. There is no sampling error on either side of the regression — 6.A2 measured what a panel does to derivatives. Employment shares then match the published unemployment rates exactly (error 1.5×10^{-14}).

Never clamp in silence

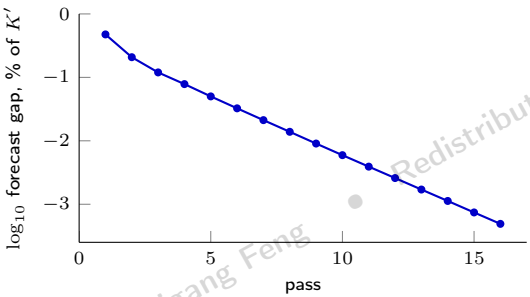
Simulated K and every forecast K' must stay strictly inside the K-grid $[30, 50]$. If not, the code stops, widens the grid and restarts. Here K stayed in $[37.08,\ 42.08]$, and the grid was never widened.

A clamp would read a policy at the wrong K and report a tidy rule anyway: this block's version of the careless bracket of 6.A2.

© Zhigang Feng • Redistribution prohibited without permission

Ledger `ks.gates`, `ks.solution` (`hist_nodes`, `K_grid_widened`).

Sixteen passes. Judge them by the forecasts, not the coefficients: intercept and slope move against each other.



Measured

16 passes, 7,360 household sweeps in all, 13 s. The forecast gap between the rule used and the rule implied fell from 0.475% of K' to 0.0005%; the coefficients moved 7×10^{-6} at the last pass.

From a stored cross-section and rule, three passes take 2 s: Lab 6 runs those.

Ledger `ks.solution.rows`, `fig_ks_convergence`, `ks.short_run`.

© Zhigang Feng · Redistribution prohibited without permission

Two Ways to Grade the Rule: One Step, or the Whole Path

Both compare the rule's forecast with the capital the simulated economy actually produced. They differ in what the rule is allowed to see.

The rule. $\log \hat{K}_{t+1} = \kappa_0(z_t) + \kappa_1(z_t) \log K_t$, with $\kappa_1 \approx 0.965$.

One step (R^2, the regression's own score). Every quarter, feed the rule the *true* K_t and predict K_{t+1}. Next quarter, start again from the true K_{t+1}. A mistake never carries over.

Den Haan's test. Feed the rule the true K_0 *once*. After that it sees only the shocks z_t and its own forecasts:

$$\hat{K}_{t+1} = H(\hat{K}_t, z_t), \qquad e_t = 100 \left| \log \hat{K}_t - \log K_t \right|.$$

This is how a household uses the rule: it forecasts many quarters ahead from today.

Why the two numbers differ

On the whole path, today's error is passed on with weight κ_1, and the new one-step error is added: $e_{t+1} \approx \kappa_1 e_t +$(one-step error). Errors can pile up to $1/(1-\kappa_1) \approx 28$ times the one-step size.

Pitfall

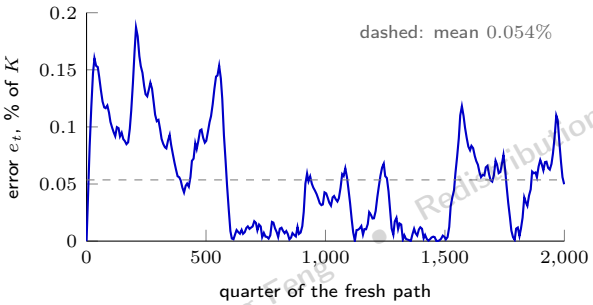
K moves slowly, so the true K_t alone predicts K_{t+1} well. R^2 is close to 1 for almost any rule and cannot tell a good rule from a poor one.

© Zhigang Feng · Redistribution prohibited without permission

Den Haan, *JEDC* 34 (2010) — the test 1.B2 promised. `fit_rule`, `rule_path` in `s06_ks.py`; ledger `ks.solution` (κ).

Den Haan's Test, Run on a Fresh Path

2,000 quarters with new shocks, the rule started once at the true K. The error rises and falls with the shocks, touches zero when it changes sign, and never drifts away.



Measured, same rule

	mean	max
one step	0.002%	0.013%
Den Haan	0.054%	0.187%

$R^2 = 0.9999985$ (recession), 0.9999991 (boom).

On average the path error is about 24 times the one-step error, close to the factor of 28 on the last frame. It is still small, at under 0.2% of K, so this rule passes.

On the fitting path (2,500 quarters): mean 0.054%, **max** 0.150%. **Ledger** `ks.diagnostics`, `ks.fig_den_haan_error`.

© Zhigang Feng · Redistribution prohibited without permission

Against the Published Benchmark

The same economy, solved by Maliar, Maliar and Valli (2010) with their own grids and simulation. The rules agree as forecasts.

	this code	MMV (2010)
κ_0, recession	0.1240	0.1221
κ_1, recession	0.9655	0.9659
κ_0, boom	0.1371	0.1363
κ_1, boom	0.9634	0.9636
mean K	39.69	39.04

Reading it

Coefficients differ in the third decimal, but as forecasts over the range K visits the two rules differ by at most 0.03% of K'. Their Den Haan error: mean 0.050%, max 0.156% over 10,000 periods.

Transcribed

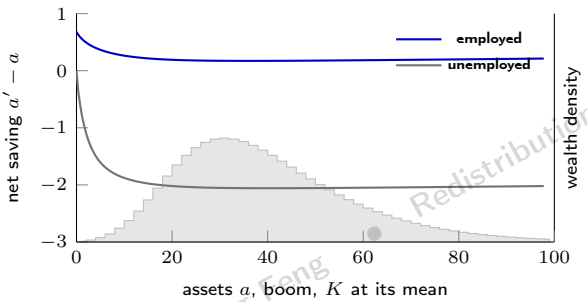
The right column is copied from the paper, not recomputed here.

© Zhigang Feng
Redistribution prohibited without permission

Maliar, Maliar & Valli, *JEDC* 34 (2010): their non-stochastic-simulation law of motion. Ledger `ks.benchmark`.

Why One Moment Is Nearly Enough

Where the wealth is, saving is a straight line in wealth with one slope. Moving wealth between such households does not change total saving.



© Zhigang Feng
Redistribution prohibited without permission

Measured

A straight line fits $a'(a)$ with mass-weighted $R^2 = 0.9999985$ for the employed and 0.999993 for the unemployed; the slopes are 0.9998 and 0.9991. Only 0.003% of households sit at the limit, where the line bends.

A property of a calibration

Not a theorem (2.B1). Fat right tails, many households at the limit, or a policy that moves them would break it.

Ledger `ks.fig_ks_policy` (`linear_fit`), `ks.diagnostics.wealth`; **grey: the time-averaged wealth distribution**.

5 · Where Moments Run Out — and What the Whole Distribution Would Cost

Adding moments buys accuracy and costs a dimension each. And a government choosing policy needs what no moment gives: how the whole distribution responds.

Three problems the mean cannot fix:

- no guidance on which moments, or what functional form;
- welfare and inequality need the shape, not two moments;
- a government searching over policies needs $\partial V/\partial \Gamma$ at every policy it might choose — aggregation was verified at one.

Keep Γ itself, then. 6.A's grid holds it in $1{,}400$ numbers. A Smolyak grid over them (5.B3): $2{,}801$ points at $\mu = 1$, $3{,}922{,}801$ at $\mu = 2$.

Count the work

Each point needs a household and a distribution: 0.12 s in 6.A. At $\mu = 2$ that is 5.3 days — for *one* evaluation of a government's objective, inside a search over policies.

Not only arithmetic

A distribution lives on a simplex, not a box; relabelling households should change nothing; simulation destroys derivatives (6.A2). 7.B4 and Session 8 take them up.

© Zhigang Feng · Redistribution prohibited without permission

Ledger `wall`: **Smolyak counts $2d+1$ and $2d^2+2d+1$, checked against every row of the Session 4 ledger**; `core.inner_solve_seconds`.

How It Works (1): VFI on (a, s, K)

```
def ks_expected_value(V, th, m0, P):
    """E[V(a', s', K') | s, K] on the asset nodes.  K' = forecast(z, K) lies between the
    K-nodes m0 and m0 + 1: read V there with weights 1 - th and th (linear in log K)."""
    Vn = (1.0 - th) * V[:, :, m0] + th * V[:, :, m0 + 1]       # (nA, 4 s', 2 z, nK)
    return np.einsum("sp,apsm->asm", P, Vn[:, :, Z_OF_S, :])  # sum over s' with P[s, s']
```

```
    for it in range(1, max_iter + 1):
        EV, V_new = ks_expected_value(V, th, m0, P), np.empty_like(V)
        for s in range(4):                                      # (employment, z) today
            for m in range(nK):                                 # K-node today
                vhat = interpolant(interp, a, EV[:, s, m])      # V between asset nodes
                obj = lambda x: u(coh[:, s, m] - x, p.gamma) + p.beta * vhat(x)
                ap[:, s, m], V_new[:, s, m], _ = golden_max(obj, p.a_min, top[:, s, m],
                                                            n_golden)
        history.append(float(np.abs(V_new - V).max()))
        V = V_new
        if history[-1] < tol:
            converged = True
            break
        for _ in range(n_howard):                               # Howard: keep a', update V
            EV = ks_expected_value(V, th, m0, P)
            for s in range(4):
                for m in range(nK):
                    vhat = interpolant(interp, a, EV[:, s, m])
                    V[:, s, m] = (u(coh[:, s, m] - ap[:, s, m], p.gamma)
                                  + p.beta * vhat(ap[:, s, m]))
        if max_seconds is not None and time.perf_counter() - t0 > max_seconds:
            break
```

© Zingales Feng · Redistribution Prohibited without permission

How It Works (2): One EGM Sweep at the Forecast K', and the Grid Guard

```
    for it in range(1, max_iter + 1):
        cn = (1.0 - th) * c[:, :, m0] + th * c[:, :, m0 + 1]   # c'(a', s', K'(z, K))
        MU = Rp[None] * cn ** (-p.gamma)                        # R' u'(c')
        Euc = np.einsum("sp,apsm->asm", P, MU[:, :, Z_OF_S, :])  # E[R' u'(c') | s]
        c_end = (p.beta * Euc) ** (-1.0 / p.gamma)             # invert the Euler equation
        a_end = (c_end + a[:, None, None] - inc[None]) / R[None]   # endogenous grid
        c_new = _back_to_grid(a, a_end.reshape(nA, -1), c_end.reshape(nA, -1),
                              Rf, incf, p.a_min)                # limit: a comparison
        c_new = np.maximum(c_new.reshape(nA, 4, nK), 1e-10)
        diff = float(np.abs(c_new - c).max())
        c = c_new
        if diff < tol:
            break
```

```
def check_rule_inside(kappa, logK):
    """The forecast must map the K grid into itself, or the household would read a
    clamped policy: stop and say so."""
    logKp = kappa[:, 0][:, None] + kappa[:, 1][:, None] * logK[None, :]
    if logKp.min() <= logK[0] or logKp.max() >= logK[-1]:
        raise KGridError(f"forecast K' in [{np.exp(logKp.min()):.2f}, "
                         f"{np.exp(logKp.max()):.2f}] reaches the K grid "
                         f"[{np.exp(logK[0]):.2f}, {np.exp(logK[-1]):.2f}]: widen it")
    return logKp
```

© Zhiyang Huang · Redistribution prohibited without permission

How It Works (3): The Loop

```
for outer in range(1, s.max_outer + 1):
    t1 = time.time()
    try:                                                # 1. household  2. simulate
        c, it, _ = solve_household_ks(kappa, p, a, logK, P, c_init=c,
                                      tol=s.tol_egm, max_iter=s.max_egm)
        sim = simulate(c, p, a, logK, Pz, P, z, lam0, h)
    except KGridError as err:                           # never clamp: widen, restart
        widened += 1
        if widened > 3:
            raise
        lo, hi = np.exp(logK[0]) * 0.85, np.exp(logK[-1]) * 1.15
        logK, c = np.linspace(np.log(lo), np.log(hi), s.n_K), None
        _say(verbose, f"   {err}\n   K grid widened to [{lo:.1f}, {hi:.1f}]")
        continue
    sweeps += it
    kappa_hat, r2 = fit_rule(sim["K"], z, s.T_burn)  # 3. regress, state by state
    change = float(np.abs(kappa_hat - kappa).max())
    rows.append(_pass_record(outer, it, change, kappa, kappa_hat, sim, r2, t1,
                             verbose))
    if change < s.tol_coef:
        break
    kappa = (1.0 - s.damping) * kappa + s.damping * kappa_hat     # 4. damped update
    lam0 = restart_distribution(sim["lam_end"], p.u[z0])
```

© Yiqiang Feng · Redistribution prohibited without permission

How It Works (4): One Quarter of the Histogram, the Regression, the Test

```
        moved = np.empty((nH, 2))
        for e in range(2):                          # a' on the histogram nodes h
            coh = (1.0 - delta + r) * h + income[e]
            ap = np.clip(coh - np.interp(h, a, ct[:, e]), p.a_min, h[-1])
            q = (ap - h[0]) / dh                    # Young's lottery, uniform grid
            j = np.minimum(q.astype(np.int64), nH - 2)
            w_hi = q - j
            moved[:, e] = (np.bincount(j, weights=lam[:, e] * (1.0 - w_hi), minlength=nH)
                           + np.bincount(j + 1, weights=lam[:, e] * w_hi, minlength=nH))
        zj = z[t + 1]                               # employment moves, given z -> z'
        lam = moved @ (P[s0:s0 + 2, 2 * zj:2 * zj + 2] / Pz[zi, zj])
```

```
def fit_rule(K, z, start):
    """Least squares of log K' on log K, separately by today's aggregate state."""
    x, y, zz = np.log(K[start:-1]), np.log(K[start + 1:]), z[start:-1]
    kappa, r2 = np.empty((2, 2)), np.empty(2)
    for zi in range(2):
        xi, yi = x[zz == zi], y[zz == zi]
        A = np.column_stack([np.ones_like(xi), xi])
        b, *_ = np.linalg.lstsq(A, yi, rcond=None)
        kappa[zi] = b
        r2[zi] = 1.0 - ((yi - A @ b) ** 2).sum() / ((yi - yi.mean()) ** 2).sum()
    return kappa, r2
```

```
def rule_path(kappa, K_start, z):
    """Iterate the perceived law forward from one K, never looking at the simulation."""
    Kf = np.empty(z.size)
    Kf[0] = K_start
    for t in range(z.size - 1):
        Kf[t + 1] = np.exp(kappa[z[t], 0] + kappa[z[t], 1] * np.log(Kf[t]))
    return Kf
```

© Zhiqiang Feng · Redistribution prohibited without permission

Takeaways

- **Part 1.** With an aggregate shock the distribution is a state. Krusell and Smith forecast it by its mean: $\log K' = \kappa_0(z) + \kappa_1(z) \log K$, a fixed point in four numbers.
- **Part 2.** The household has four states. VFI reads V at the forecast K' (linear in $\log K$, then in a): 1,382 sweeps plain, 62 maximizations with Howard. EGM: 0.31 s. They agree to 0.1% where households are.
- **Part 3.** Solve, push a histogram, regress, damp. No panel, no silent clamp.
- **Part 4.** 16 passes; $R^2 = 0.999999$ says little, Den Haan's error (0.19% at worst on a fresh path) says more. The published rule agrees as a forecast.
- **Part 5.** One moment works because saving is linear in wealth where the wealth is. A government needs the whole distribution: 3.9 million Smolyak points at $\mu = 2$.

© Zhigang Feng
Redistribution prohibited without permission

Bridge: One Surprise Instead of a Shock That Keeps Coming

Krusell–Smith's distribution moves because the aggregate shock never stops. Remove the shock, and change one thing once: a tax, announced today, for ever.

After that surprise nothing is random in the aggregate. Prices become a *sequence* rather than a function of the distribution — and the problem becomes a path to compute.

Next: 6.B2

An MIT shock: first in the representative-agent growth model, solved three ways; then the same shock in Block A's economy, with a distribution; then who gains. And the life cycle, where cohorts are the distribution.

© Zhigang Feng · Reproduction prohibited without permission